K Graph Centers

Wei Chen, Xiaohui Liu, Man Qiu, *Graduate Student Member, IEEE*, and Zhenning Xiong

***Abstract*—K-means is a powerful unsupervised clustering method for regularly structured data, relying fundamentally on geometric cluster centers. However, extending this paradigm to graph-structured data remains challenging. We propose "K-Graph-Centers" — a novel method that defines the graph cluster via "graph center", represented as an N-dimensional vector minimizing the sum of weighted topological distances within its cluster. We introduce an efficient power iteration scheme to compute graph centers by: 1) diffusing affinity through normalized adjacency matrix; 2) enforcing sparsity via row-wise discretization. Cluster assignments are derived from the converged graph centers, similar with K-means. K-Graph-Centers method exhibits rather lower computational complexity and fast convergence speed, compared to traditional spectral-based methods. The experimental results verify its time efficiency and clustering performance in common metrics.**

***Index Terms*—K-graph-centers, virtual graph center, spectral clustering, graph cohesiveness, K-means.**

I. Introduction

In THE vast landscape of unsupervised clustering, K-means [1] stands as one of the most fundamental, widely recognized, and practically implemented algorithms. Its mission is simple but powerful: to partition a given dataset into K distinct, non-overlapping clusters. Its core operation aims to minimize intra-cluster similarity while maximize inter-cluster similarity, where "similarity" is typically measured using the Euclidean distance. This algorithm implicitly assumes that clusters are isotropic distributions centered around their means, thereby forming a convex geometric structure.

The superiority of K-means lies in its simplicity and efficiency, achieved through an iterative refinement process. Firstly, it initializes K cluster centers, which acts a critical step with significant impact on the final result. Then, at assignment step, cluster belonging for each point is assigned by comparing its distances to the K current centers. As a consequence, each point belongs exclusively to one cluster, achieving a hard classification. At mean calculation step, center location for each cluster is updated referring to current cluster assignments of all points. Assignment step and mean calculation step are conducted alternately per iteration, until the convergence is reached. The algorithm termination occurs when label vector stabilizes, consecutive mean difference falls below some threshold, or maximum iteration elapses.

The simplicity and efficiency of K-means and its variants [2], [3], [4] have led to wide applications across numerous domains, such as customer segmentation [5], image hashing [6], light field processing [7], attack detection [8], biomedical summarization [9], etc. K-means excels primarily due to its computational efficiency and easy implementation. The algorithm typically converges quickly in practice, with a time complexity $O(tKN)$ (t represents iteration counts, K indicates cluster number, N is the dataset size). This linear time complexity makes it highly scalable to large datasets compared to its competitors. Its core logic is intuitive and easy to understand, implement, and debug. For datasets whose underlying clusters are approximate Gaussian distributive, K-means often produces accurate and interpretable results efficiently.

Though achieved large progress, K-means still possesses significant limitations, such as the need to predefined cluster number, sensitivity to initialization, and incapability to handle non-convex dataset. Especially the last one, it severely hinders the further application of this algorithm. To address the critical limitation, researchers have developed several alternative clustering methods that fundamentally circumvent structural limitations inherent in dataset morphology. Density-based methods [10], [11], [12], overcome such a constraint by defining clusters as dense regions separated by sparse areas, naturally identifying irregular structures and outliers without requiring predefined cluster number. However, their effectiveness relies critically on sensitive parameters (e.g., neighborhood radius and minimum points), struggles with varying densities. Grid-based approaches such as [13], [14], [15], partition the data space into cells, enabling fast processing by operating on quantized units rather than individual points. Yet, their rigid axis-aligned partitioning generates discontinuous axis-aligned boundaries, lose granularity at coarse resolutions, and prove ineffective for correlated dimensions. Hierarchical methods, including [16], [17], [18], etc., build multi-level cluster structures through iterative merging or splitting, providing flexibility in cluster extraction without fixing K beforehand. Nevertheless, their high

Received 12 September 2025; revised 14 April 2026; accepted 28 May 2026. Date of publication 8 June 2026; date of current version 8 August 2026. This work is supported by NSFC Major Research Plan on West-Pacific Earth System Multispheric Interactions under Grant 92358303. Recommended for acceptance by Y. Yuan. *(Corresponding author: Wei Chen.)*

Wei Chen is with the School of Software and Internet of Things Engineering, Jiangxi University of Finance and Economics, Nanchang City 330013, China (e-mail: chenwei@jxufe.edu.cn).

Xiaohui Liu is with the Key Laboratory of Data Science in Finance and Economics & Philosophy and Social Sciences Laboratory of Data Science in Finance and Economics at the Ministry of Education, Jiangxi University of Finance and Economics, Nanchang City 330013, China (e-mail: liuxiaohui@jxufe.edu.cn).

Man Qiu is with Southeast University, Nanjing City 210096, China (e-mail: 220256847@seu.edu.cn).

Zhenning Xiong is with the Jiangxi University of Finance and Economics, Nanchang 330013, China (e-mail: 2202004145@stu.jxufe.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3701456

computational complexity (often O(N^2)) makes them impractical for large-scale datasets, and early-stage linkage decisions (always susceptible to noise) propagate irreversibly through the hierarchy, compromising final results.

These inherent constraints in density, grid, and hierarchical techniques drive our attention to spectral-based method, aiming to achieve more general clustering. Representing data points as vertices and their relationships as weighted edges, this paradigm transforms clustering into a graph partitioning problem. These spectral-based algorithms apply the eigenvectors of Laplacian matrix to reveal low-dimensional embedding that exposes nonlinear structures. They excel by fitting arbitrary topology – clusters manifest as strongly connected components regardless of geometric form. They integrate both local neighborhood relationships (via edge weights) and global data topology through graph connectivity, inherently adapt to density variations by leveraging edge weights within subgraphs. They also have a solid mathematical foundation via spectral graph theory. Beyond the rigidity from absolute spatial structure, spectral-based clustering effectively mitigates the core geometric plaguing K-means and its alternatives, establishing itself as a powerful tool for modern exploratory data analysis.

RCUT [19] and NCUT [20] act the most frequently two algorithms, which also are the origins of most their variants. The common procedure of these two algorithms can be described as follows. Typically, they start by constructing a symmetric weighted adjacency matrix, where each elements encodes pairwise similarity between data points. This similarity metric is frequently derived from Gaussian similarity measurements. Subsequently, these algorithms compute the first K eigenvectors of the associated graph Laplacian matrix. After that, these eigenvectors are arranged as columns to form an indicator matrix. To implement clustering, they finally identify the row vectors of this indicator matrix as K distinct groups, employing standard techniques such as K-means or spectral rotation [21].

However, these two classical spectral-based methods also associate several constraints, such as relative high computational cost, irreversible information loss from additional discretization on the indicator matrix, and severe reliance on the similarity measurement, etc. Lots of variants have been proposed continuously in recent years. To address last two inherent challenges in spectral clustering, researchers have pursued three primary optimization strategies. The first approach focuses on discrete optimization by directly incorporating discretization constraints. This includes techniques such as discrete transformations [22], [23], [24] and spectral rotation methods that jointly optimize eigenvector computation and cluster indicator discretization [21]. The second strategy centers on similarity matrix learning, where main progress has been made through novel constraint formulations. Notable examples lie on simultaneously learning similarity structures and deciding cluster assignments, which are achieved via minimizing objective functions that combine graph fidelity terms with low-rank constraints, as implemented in Clustering with Adaptive Neighbors (CAN) [25], Projected Clustering with Adaptive Neighbors (PCAN) [25], and Constrained Laplacian Rank (CLR) [26]. Innovations from other researchers also develop similar ideas [27], [28], [29]. The third direction integrates these approaches by collaboratively optimizing both the similarity matrix and discrete cluster indicator matrix. This unified methodology is presented in Discrete Optimal Graph Clustering (DOGC) [30] and Direct Spectral Clustering (DSC) [31]. Conclusively, these surveyed techniques demonstrate significant advancements in resolving the core limitations of these spectral clustering methods.

Time cost of these spectral-based methods also badly limits their practical applications. For most of these algorithms, their solving procedures commonly involve an iterative computation of all eigenvectors or the first K eigenvectors, which introduces a fundamental time complexity of $O(N^3)$ or $O(tKN^2)$. Such a computational cost is not affordable in many cases, particularly in their variants, where the additional iteration enclosing the eigen decomposition would be introduced frequently. This drawback is crucial but rarely addressed.

As a consequence, it is natural to combine the intuitiveness and efficiency of K-means method with the representation and capability of graph structure, to form an innovative and superior clustering framework. We must define the "graph center" concept, which greatly differs from the naive center in Euclidean space, since graph is described in relative relationship. Additionally, the calculation for graph center must be reformed from bottom. Due to the deep gap between the two data structures, it introduces great challenge for us to transfer the K-means paradigm into graph.

In this paper, inspired by K-means, we propose a spectral-based clustering algorithm named "K-Graph-Centers". Firstly, the concepts of "graph cohesiveness" and "graph center" are defined, aiming to describe the compact level and centroid location of the graph respectively. After modeling the graph cohesiveness feature, we derive the quantitative expression and corresponding solution for graph center. Based on this result and referred by K-means, we invent the "K-Graph-Centers" method. In the first step, it updates the K subgraph centers of the mixture graph by a power iteration conducted on the normalized adjacency matrix. Then a discretization is operated upon the updated graph center vectors to determine the cluster memberships for the graph vertices, which is similar to cluster assignment in K-means, but still with tiny difference. These two steps progress alternately, until the convergence is reached.

Summarily, this new algorithm "K-Graph-Centers" contributes as follows:

1) Introduces the innovative concepts, and derives the explicit mathematical solutions for "graph cohesiveness" and "graph center". "Graph cohesiveness" is proposed to quantitatively measure the graph compactness, and "graph center" positions the graph topological structure and records the vertex contributions to the graph cohesiveness;

2) Transfers K-means paradigm to analyze graph data, provides a simple, intuitive and effective method for graph clustering successfully. Basically, it designs a novel spectral embedding composed by affinity diffusion and graph center discretization;

3) Presents an alternate continuous-discrete-coupled scheme for graph optimization, and addresses the computational bottleneck in current spectral clustering by its nature sparseness;

4) The experiments conducted on synthetic datasets and real-world datasets exhibit the capability and superiority of "K-Graph-Centers". Notably, this algorithm outstands significantly in computational efficiency against its state-of-the-art competitors.

The remaining of this paper is organized as follows: in Section II, we introduce the novel "K-Graph-Centers" method, including the definition and solution of graph cohesiveness and graph center, detailed algorithms description, convergence and complexity analysis; Section III exhibits the feasibility test on a small graph, then the comparative experiments on synthetic and real-world datasets with other similar type of algorithms; in Section IV, we conclude this work and forecast the future study.

II. Proposed New Algorithm

A. Graph Cohesiveness and Graph Center

Suppose an undirected graph $\mathbb{G}(\mathbf{V},\mathbf{E})$ of N vertices, M edges. Note $\mathcal{C}$ as the graph center of graph $\mathbb{G}$, and its inverse distances to all vertices are expressed in a nonnegative vector $\mathbf{x}$. Therefore, we can use vector $\mathbf{x}$ to represent graph center C. To act as a graph center, the total distance from C to all the N vertices in $\mathbb{G}$ must be minimized. To satisfy this property, on one hand we must constrain its distances to vertices with larger weights as smaller as possible, and vice versa, so that it is reasonable to maximize:

$$\begin{aligned}&\sum_{i=1}^{N}\sum_{j=1}^{N} w_{ij}x_i x_j\\ &= \begin{bmatrix} x_1\, x_2\, \cdots\, x_N \end{bmatrix} \begin{bmatrix} 0 & w_{12} & w_{13} & \cdots & w_{1N}\\ w_{21} & 0 & w_{23} & \cdots & w_{2N}\\ w_{31} & w_{32} & 0 & \cdots & w_{3N}\\ \vdots & \vdots & \vdots & \ddots & \vdots\\ w_{N1} & w_{N2} & w_{N3} & \cdots & 0 \end{bmatrix} \begin{bmatrix} x_1\\ x_2\\ x_3\\ \vdots\\ x_N \end{bmatrix}\\ &= \mathbf{x}^T\mathbf{W}\mathbf{x}. \end{aligned} \tag{1}$$

where $\mathbf{W}$ is the symmetric adjacency matrix of $\mathbb{G}$, $w_{i,j}$ is its element in i^{th} row and j^{th} column, x_i and x_j are the i^{th} and j^{th} elements in $\mathbf{x}$. On the other hand, behaving as a center, its distances to closer vertices must also be closer, that is to minimize such a term:

$$\begin{aligned}\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} w_{i,j}(x_i - x_j)^2 &= \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} w_{i,j}(x_i^2 - 2x_i x_j + x_j^2)\\ &= \sum_{i=1}^{N}\left(\sum_{j=1}^{N} w_{i,j}\right)x_i^2 - \sum_{i=1}^{N}\sum_{j=1}^{N} w_{ij}x_i x_j\\ &= \sum_{i=1}^{N} d_i x_i^2 - \mathbf{x}^T\mathbf{W}\mathbf{x}\\ &= \mathbf{x}^T\mathbf{D}\mathbf{x} - \mathbf{x}^T\mathbf{W}\mathbf{x}\\ &= \mathbf{x}^T\mathbf{L}_s\mathbf{x} \end{aligned} \tag{2}$$

where $\mathbf{L}_s = \mathbf{D} - \mathbf{W}$, $\mathbf{D}$ is the diagonal degree matrix, with its diagonal element $d_i = \sum_{j=1}^{N} w_{i,j}$. Therefore, as a summary, to determine the location of graph center $\mathcal{C}$, the quotient of (2) and (1) with the constraint $\mathbf{x} > 0$ must be minimized. However, to promise the quotient nonnegative and the divider nonzero (here bipartite graphs are not considered), we replace (1) by:

$$\begin{aligned}\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} w_{i,j}(x_i + x_j)^2 &= \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} w_{i,j}(x_i^2 + 2x_i x_j + x_j^2)\\ &= \sum_{i=1}^{N}\left(\sum_{j=1}^{N} w_{i,j}\right)x_i^2\\ &\quad + \sum_{i=1}^{N}\sum_{j=1}^{N} w_{ij}x_i x_j\\ &= \mathbf{x}^T\mathbf{D}\mathbf{x} + \mathbf{x}^T\mathbf{W}\mathbf{x}\\ &= \mathbf{x}^T\mathbf{L}_a\mathbf{x}, \end{aligned} \tag{3}$$

where $\mathbf{L}_a = \mathbf{D} + \mathbf{W}$. This term is equivalent to (1) on maximizing the self similarity of $\mathbf{x}$ on graph $\mathbb{G}$, since $\mathbf{x}^T\mathbf{D}\mathbf{x}$ keeps constant when $\mathbf{x}$ is normalized.

So finally, we get this objective with a generalized Rayleigh quotient form to minimize:

$$f_c(\mathbf{x}) = \frac{\sum_{i=1}^{N}\sum_{j=1}^{N} w_{i,j}(x_i - x_j)^2}{\sum_{i=1}^{N}\sum_{j=1}^{N} w_{i,j}(x_i + x_j)^2} = \frac{\mathbf{x}^T\mathbf{L}_s\mathbf{x}}{\mathbf{x}^T\mathbf{L}_a\mathbf{x}},\ \text{s.t. } \mathbf{x} > 0. \tag{4}$$

We can name the feature described in (4) as "graph cohesiveness", due to its value quantitatively expresses the closeness of graph vertices to their graph center. This expression is simple and thus convenient for further processing. In fact, to obtain the target $\mathbf{x}$, it equals to minimize this term [32]:

$$f_c(\mathbf{x}) = \mathbf{x}^T\mathbf{L}_c\mathbf{x},\ \text{s.t. } \mathbf{L}_c = \mathbf{L}_a^{-1}\mathbf{L}_s \text{ and } \mathbf{x} > 0. \tag{5}$$

Set $\mathbf{v}$ as one eigenvector of $\mathbf{L}_c$, and λ as the corresponding eigenvalue, we have:

$$\begin{aligned}&\mathbf{L}_c\mathbf{v} = \mathbf{L}_a^{-1}\mathbf{L}_s\mathbf{v} = \lambda\mathbf{v},\\ &\mathbf{L}_s\mathbf{v} = \lambda\mathbf{L}_a\mathbf{v},\\ &(\mathbf{D} - \mathbf{W})\mathbf{v} = \lambda(\mathbf{D} + \mathbf{W})\mathbf{v},\\ &\mathbf{D}^{-1}\mathbf{W}\mathbf{v} = \frac{1-\lambda}{1+\lambda}\mathbf{v}, \end{aligned} \tag{6}$$

which means that we can calculate the eigenvector of $\mathbf{D}^{-1}\mathbf{W}$ instead. Consequently, its time-cost is reduced greatly since we have no need to calculate the inverse of $\mathbf{L}_a$. To obtain the target eigenvector $\mathbf{v}$, we refer to the power iteration method, that is:

$$\mathbf{v} = \mathbf{D}^{-1}\mathbf{W}\mathbf{v}. \tag{7}$$

Since $\mathbf{v}$ must be a positive vector, if applying a positive initialization and a normalization after each iteration, $\mathbf{v}$ will converge the eigenvector which corresponds to eigenvalue 1 from the power iteration method for $\mathbf{D}^{-1}\mathbf{W}$, with a form of $\mathbf{v} = (\cdots, \frac{1}{\sqrt{N}}, \cdots)^T$. This result is correct but useless, since the obtained eigenvector is uniform thus loses distinctiveness

in its elements, we can not explore it for further applications. In other words, a uniform $\mathbf{v}$ indicating the graph center has equal distances to all graph vertices, thus brings us no useful information. So single center graph for the whole graph will fail to express the graph structure. However, combined with some additional operations, searching multiple graph centers for the subgraphs in a mixture graph still works. The key is to isolate the mutual effect from other subgraphs, we will show how to realize it in later section.

B. K-Graph-Centers

From the perspective of center updating in K-means, we review the iteration in (7), the i^{th} element in $\mathbf{v}$ can be expressed as:

$$v_i^{(t+1)} = \sum_{j=1}^{N} \frac{w_{i,j}}{d_{i,i}} v_j^{(t)}. \tag{8}$$

where $d_{i,i} = \sum_{j=1}^{N} w_{i,j}$. That is to say the inverse distance between i^{th} vertex to the $(t+1)^{th}$ graph center $\mathcal{C}^{(t+1)}$ (noted as $v_i^{(t+1)}$), is defined by the weighted inverse distances from its neighbors to previous graph center $\mathcal{C}^{(t)}$, with their normalized edge weights as the coefficients. This updating operation acts the same with centroid calculation in the particle system, for which $\mathbf{v}$ can represent "graph center" $\mathcal{C}$.

Though we have derived the single graph center for single graph, when confronting with mixture graph composed by several components, there exists multiple graph centers. We must find a way to calculate all the subgraph centers. Note cluster number K, we assemble these K centers together to form a nonnegative assemble subgraph center matrix $\mathbf{X}^{N\times K}$. Therefore the iteration to determine $\mathbf{X}$ can be expressed as:

$$\mathbf{X} = \mathbf{D}^{-1}\mathbf{W}\mathbf{X}. \tag{9}$$

However, as mentioned before, when the iteration progresses, the affinity diffuses and spreads to the whole graph. If without any stop operation, $\mathbf{X}$ will converge to a uniform matrix unfortunately. This result is not expected by us. The critical issue is how to hinder the affinity diffusion in $\mathbf{X}$ and constrain the effect only applied within the subgraph. Also we must normalize the column vectors in X after the updating operation to prevent them from vanishing or blowing up during the iteration. Here both L_1 and L_2 normalization approaches are proper. And we apply L_2 normalization in all the following tests.

$\mathbf{X}$ contains K vectors which describe the inverse distances from all vertices to the K subgraph centers, thus referring to K-means, after each iteration, we can identify and update the cluster assignments for all vertices by comparing the row elements in $\mathbf{X}$. We force all elements except the maximum to zero for each row vector of $\mathbf{X}$. That is, for $i = 1, 2, 3, \ldots, N$ and $j = 1, 2, \ldots, K$, we discretize row vectors of $\mathbf{X}$ as:

$$x_{i,j} = \begin{cases} x_{i,j} & \text{if } j = \arg\max\limits_{1\le k\le K} x_{i,k}, \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

This discretization operation can not only avoid $\mathbf{X}$ converging to a uniform matrix, but also maintain mutual orthogonality among its column vectors. This operation is similar with the binarization in K-means, where the cluster assignment is determined on the one-hot vector obtained by discretizing the distance vector from target data point to K previous centers. The difference is that, cluster indicators and inverse distances are fused within a single vector in our algorithm, that is to say, the non-zero elements in these column vectors encode not only the inverse distances, but also the cluster indicators. The discretization successfully stops the affinity diffusion effect in the subgraph centers, thereby avoiding uniform vectors and maintaining distinguishability. In such a way, at each iteration, the new subgraph centers only record strong interactions within the same subgraph, achieving mutual effect isolation. In fact, the total value of a certain element in each subgraph center originates from two sources: nodes within the local subgraph and nodes of neighbor subgraphs. In traditional power iteration, the uniform vector is formed under the effects both from these two sources. while in our case, the discretization operation cuts off the contributions from neighbor subgraphs, thus distinguishability of these nonzero elements in the subgraph center emerges.

Algorithm 1: K-Graph-Centers.

1: **Input:** adjacency matrix $\mathbf{W}$ and cluster number K;
2: Initialize $\mathbf{X}^{(0)}$, calculate $\mathbf{D}$, note $\bar{\mathbf{W}} = \mathbf{D}^{-1}\mathbf{W}$ and set ϵ;
3: $t = 0$;
4: **do**
5: $\mathbf{X}^{(t+1)} = \bar{\mathbf{W}}\mathbf{X}^{(t)}$;
6: Normalize all the column vectors of $\mathbf{X}^{(t+1)}$;
7: Discretize all the row vectors of $\mathbf{X}^{(t+1)}$;
8: $t = t + 1$;
9: **while** $\|\mathbf{X}^{(t+1)} - \mathbf{X}^{(t)}\|_F^2 > \epsilon$
10: Identify the cluster label vector $\mathbf{h}$ based on final $\mathbf{X}$;
11: **Output:** Cluster label vector $\mathbf{h}$.

Cluster label assignment again relies on the final assemble subgraph center matrix $\mathbf{X}$. A simple hard classification can be conducted as: for $i \in 1, 2, \ldots, N$, the label h_i of i^{th} vertex is

$$h_i = \arg\max_{j\in\{1,2,\ldots,K\}} x_{i,j}; \tag{11}$$

or h_i equals to the nonzero-element index in i^{th} row of $\mathbf{X}$.

Since both graph center updating and cluster label assignment in this algorithm are similar to those in K-means, we name this algorithm "K-Graph-Centers".

About the convergence judgment, we can inspect the difference between two successive $\mathbf{X}$, or a faster approach is to monitor the changes of two consecutive label vectors during the iteration. The complete procedure of this algorithm is expressed in Algorithm 1, which indicates "K-Graph-Centers" is rather simple and intuitive.

C. Computational Complexity Analysis

By analyzing Algorithm 1, we can conclude that the time complexity of "K-Graph-Centers" is $O(N^2)$, or specifically, is $O(tN^2)$, where t is the iteration number. One might question why the time complexity is not $O(tKN^2)$, given the dimension of $\mathbf{X}$ is $N \times K$. In fact, due to the discretization

operation, there is only N non-zero elements in $\mathbf{X}$, but not $K * N$. Therefore, "K-Graph-Centers" algorithm achieves a low time complexity due to sparseness of $\mathbf{X}$. And from practical tests, to reach convergence, the required iteration number t is also small in this algorithm. Spectral-based methods are the closest and best competitors of our algorithm, whose time complexities are all higher than this. Specifically, for NCUT and RCUT, their time complexity are $O(N^3)$. While the variants of NCUT and RCUT often carefully optimize eigen decomposition by applying Lanczos method [33], consequently reducing its complexity to $O(cKN^2)$, where c is the iteration number of Lanczos method. Then commonly these variants enclose this eigen decomposition as their inner function, and conduct their own modifications outside. As a result, the lower bound of time complexities in these variants takes $O(tcKN^2)$, which is much higher than ours.

When considering a sparse graph, "K-Graph-Centers" exhibits more significant efficiency. In the updating operation $\mathbf{X}^{(c+1)} = \bar{\mathbf{W}}\mathbf{X}^{(c)}$, since $\bar{\mathbf{W}} = \mathbf{D}^{-1}\mathbf{W}$ is sparse, with only $2M$ non-zero elements, and $\mathbf{X}^{(c)}$ also is sparse after discretization, totally only N non-zero elements, thus this multiplication operation exhibits a time complexity of $O(2M)$. The time complexity of normalization for column vectors of $\mathbf{X}$ is still $O(KN)$. The discretization operation, takes at most KN computations. However, because $\bar{\mathbf{W}}$ and $\mathbf{X}^{(c)}$ are both sparse, then after the updating, $\mathbf{X}^{(c+1)}$ also would probably be sparse. As a result, the computation cost of this discretization operation is always far lower than KN. Therefore, the upper bound of total time complexity for once iteration in "K-Graph-Centers" is just $O(2M + 2KN)$. Set total iteration number as t and assume that $KN \ll M$, "K-Graph-Centers" features a time complexity of $O(2tM)$, which is linear with the number of graph edges and independent with cluster number K.

In summary, the time efficiency of "K-Graph-Centers" is excellent compared to common spectral-based clustering methods in theoretical. The practical results demonstrated in later experiment section also verify this conclusion.

D. Convergence Determination and Initialization Discussion

As mentioned before, our real target in this algorithm is to minimize such a term:

$$f_c(\mathbf{X}) = \sum_{i=1}^{K} \mathbf{x}_i^T \mathbf{L}_c \mathbf{x}_i, \tag{12}$$

where $\mathbf{x}_i$ is the i^{th} column vector of $\mathbf{X}$. Since $\mathbf{L}_c = \mathbf{L}_a^{-1}\mathbf{L}_s$ and as our derivation, it is equal to maximize $\sum_{i=1}^{K} \mathbf{x}_i^T \mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i$.

For Algorithm 1, we can observe that mainly two phases are contained in the loop: one to update $\mathbf{X}$, the other to discretize $\mathbf{X}$. In updating of $\mathbf{X}$, for its column vector $\mathbf{x}$, we have:

$$\mathbf{x} = a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \cdots + a_i\mathbf{v}_i + \cdots + a_N\mathbf{v}_N, \tag{13}$$

where $\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_i, \ldots, \mathbf{v}_N\}$ is the eigenvectors of $\mathbf{D}^{-1}\mathbf{W}$ corresponding to the eigenvalues $\lambda_1, \lambda_2, \cdots, \lambda_i, \cdots, \lambda_N$, with $\lambda_1 > \lambda_2 > \cdots > \lambda_i > \cdots \lambda_N$, and $a_1, a_2, \ldots, a_i, \ldots, a_N$ are the coefficients. It indicates that $\mathbf{x}$ can be represented by the basis $\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_i, \ldots, \mathbf{v}_N\}$. Then at t^{th} updating and normalizing (Lines 5 and 6in Algorithm 1):

$$\begin{aligned}\hat{\mathbf{x}}^{(t)} &= \lambda_1(a_1^{(t-1)}\mathbf{v}_1 + a_2^{(t-1)}\frac{\lambda_2}{\lambda_1}\mathbf{v}_2 \\ &\quad + \cdots + a_i^{(t-1)}\frac{\lambda_i}{\lambda_1}\mathbf{v}_i + \cdots)/||\hat{\mathbf{x}}^{(t)}||_2 \\ &= a_1^{(t)}\mathbf{v}_1 + a_2^{(t)}\mathbf{v}_2 + \cdots + a_i^{(t)}\mathbf{v}_i + \cdots,\end{aligned} \tag{14}$$

where $\hat{\mathbf{x}}^{(t)}$ denotes the vector applied with updating operation and normalizing operation, but without the discretization operation, $a_1^{(t)}, a_2^{(t)}, \ldots, a_N^{(t)}$ is the new coefficients after t^{th} updating and normalizing.

We can find the coefficient $a_1^{(t)}$ of $\mathbf{v}_1$ in $\hat{\mathbf{x}}^{(t)}$ increases as the iteration goes on, since $\lambda_1 > \lambda_2 > \cdots > \lambda_i > \cdots \lambda_N$. Eventually we have:

$$\begin{aligned}&\left(\hat{\mathbf{x}}^{(t)}\right)^T \mathbf{D}^{-1}\mathbf{W}\hat{\mathbf{x}}^{(t)} \\ &= (\alpha_1^{(t)})^2\lambda_1 + (\alpha_2^{(t)})^2\lambda_2 + \cdots + (\alpha_i^{(t)})^2\lambda_i + \cdots \\ &\geq (\alpha_1^{(t-1)})^2\lambda_1 + (\alpha_2^{(t-1)})^2\lambda_2 + \cdots + (\alpha_i^{(t-1)})^2\lambda_i + \cdots \\ &= (\mathbf{x}^{(t-1)})^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}^{(t-1)}.\end{aligned} \tag{15}$$

Considering all column vectors together, we have:

$$\sum_{i=1}^{K}\left(\hat{\mathbf{x}}_i^{(t)}\right)^T \mathbf{D}^{-1}\mathbf{W}\hat{\mathbf{x}}_i^{(t)} \geq \sum_{i=1}^{K}\left(\mathbf{x}_i{}^{(t-1)}\right)^T \mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i{}^{(t-1)}. \tag{16}$$

Thus we can conclude that, the updating and normalizing operations promise the loss function of (12) doesn't increase.

Then we consider the discretization phase. For each row of $\mathbf{X}$, this operation keeps only the maximum element, but forces the remaining $K-1$ ones to zero. Here we must consider the collective effect from this discretization and previous normalization (note as pre-normalization, shown in Line 6 in Algorithm 1) operations. For convenience, we can assume a post normalization operation added after the discretization for comparison. Such an additional normalization would not affect final results but help to our proof as demonstrated in (17) and (18). Note vector $\mathbf{v}$ after discretization, $\mathbf{v}_P$ after the post normalization, we compare the obtained two vectors after the diffusion under two conditions: with and without post normalization. For the first condition with post normalization:

$$\begin{aligned}&\mathbf{v}_P = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}, \text{ post-normalization;} \\ &\mathbf{v}_{P,D} = \mathbf{D}^{-1}\mathbf{W}\mathbf{v}_P = \mathbf{D}^{-1}\mathbf{W}\frac{\mathbf{v}}{||\mathbf{v}||_2}, \text{ diffusion;} \\ &\mathbf{v}_{P,D,p} = \frac{\mathbf{v}_{P,D}}{||\mathbf{v}_{P,D}||_2} = \frac{\mathbf{D}^{-1}\mathbf{W}\mathbf{v}}{||\mathbf{D}^{-1}\mathbf{W}\mathbf{v}||_2}, \text{ pre-normalization.}\end{aligned} \tag{17}$$

And similarly, for the second condition associated without post normalization, we obtain these relationships:

$$\begin{aligned}&\mathbf{v}_D = \mathbf{D}^{-1}\mathbf{W}\mathbf{v}, \text{ diffusion;} \\ &\mathbf{v}_{D,p} = \frac{\mathbf{v}_D}{||\mathbf{v}_D||_2} = \frac{\mathbf{D}^{-1}\mathbf{W}\mathbf{v}}{||\mathbf{D}^{-1}\mathbf{W}\mathbf{v}||_2}, \text{ pre-normalization.}\end{aligned} \tag{18}$$

Comparing (17) with (18), we find that $\mathbf{v}_{P,D,p} = \mathbf{v}_{D,p}$, which indicates the additional post-normalization brings no change for the result. However, it will make our proof easier.

The discretization and additional post-normalization operation collectively act as a polarization and amplification operation. Note S_i the current vertex set of i^{th} subgraph (also the vertex set of non-zero elements in $\mathbf{x}_i$ after the discretization), subtract normal of each $\mathbf{x}_i$ by $\sum_{i=1}^{K} \mathbf{x}_i^T \mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i$ to obtain this form:

$$\begin{aligned}&\sum_{i=1}^{K}(\mathbf{x}_i^{(t)})^T\mathbf{x}_i^{(t)} - \sum_{i=1}^{K}(\mathbf{x}_i^{(t)})^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i^{(t)}\\ &\quad= \frac{1}{2}\sum_{i=1}^{K}\sum_{l,m\in S_i}\frac{w_{l,m}}{d_{l,l}}(x_{l,i}^{(t)} - x_{m,i}^{(t)})^2,\end{aligned} \tag{19}$$

where we know ${\mathbf{x}_i^{(t)}}^T\mathbf{x}_i^{(t)} = 1$ due to the post normalization. Note the centroid of each column vector of $\mathbf{X}$ as $x_{c,i}^{(t)}$, we can rewrite (19) as:

$$\begin{aligned}&\sum_{i=1}^{K}\sum_{l,m\in S_i}\frac{w_{l,m}}{d_{l,l}}(x_{l,i}^{(t)} - x_{m,i}^{(t)})^2\\ &= \sum_{i=1}^{K}\sum_{l,m\in S_i}\frac{w_{l,m}}{d_{l,l}}(x_{l,i}^{(t)} - x_{c,i}^{(t)} + x_{c,i}^{(t)} - x_{m,i}^{(t)})^2\\ &= \sum_{i=1}^{K}\sum_{l,m\in S_i}\frac{w_{l,m}}{d_{l,l}}((x_{l,i}^{(t)} - x_{c,i}^{(t)})^2 + (x_{m,i}^{(t)} - x_{c,i}^{(t)})^2).\end{aligned} \tag{20}$$

After the discretization and post normalization, the nodes with non-zero elements would distribute closer to its centroid due to tail elements are removed, that is to say:

$$\begin{aligned}&\sum_{l,m\in S_i}\frac{w_{l,m}}{d_{l,l}}((x_{l,i}^{(t)} - x_{c,i}^{(t)})^2 + (x_{m,i}^{(t)} - x_{c,i}^{(t)})^2)\\ &\le \sum_{l,m\in \hat{S}_i}\frac{w_{l,m}}{d_{l,l}}((\hat{x}_{l,i}^{(t)} - \hat{x}_{c,i}^{(t)})^2 + (\hat{x}_{m,i}^{(t)} - \hat{x}_{c,i}^{(t)})^2),\end{aligned} \tag{21}$$

where $\hat{S}_i$ is the vertex set of non-zero elements in $\hat{\mathbf{x}}_i$ before the discretization. Referring to (19), we have:

$$\begin{aligned}&\sum_{i=1}^{K}(\mathbf{x}_i^{(t)})^T\mathbf{x}_i^{(t)} - \sum_{i=1}^{K}(\mathbf{x}_i^{(t)})^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i^{(t)}\\ &\le \sum_{i=1}^{K}(\hat{\mathbf{x}}_i^{(t)})^T\hat{\mathbf{x}}_i^{(t)} - \sum_{i=1}^{K}(\hat{\mathbf{x}}_i^{(t)})^T\mathbf{D}^{-1}\mathbf{W}\hat{\mathbf{x}}_i^{(t)}.\end{aligned} \tag{22}$$

Therefore, this relation can be obtained:

$$\sum_{i=1}^{K}(\mathbf{x}_i^{(t)})^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i^{(t)} \ge \sum_{i=1}^{K}(\hat{\mathbf{x}}_i^{(t)})^T\mathbf{D}^{-1}\mathbf{W}\hat{\mathbf{x}}_i^{(t)}. \tag{23}$$

Combined (16) with (23), we can conclude that:

$$\begin{aligned}\sum_{i=1}^{K}\left(\mathbf{x}_i^{(t)}\right)^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i^{(t)} &\ge \sum_{i=1}^{K}\left(\hat{\mathbf{x}}_i^{(t)}\right)^T\mathbf{D}^{-1}\mathbf{W}\hat{\mathbf{x}}_i^{(t)}\\ &\ge \sum_{i=1}^{K}\left(\mathbf{x}_i{}^{(t-1)}\right)^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i{}^{(t-1)}.\end{aligned} \tag{24}$$

Consequently, the convergence of "K-Graph-Centers" has been verified.

We need to initialize all the elements $\mathbf{X}$ to nonnegative values. "K-Graph-Centers" algorithm is also sensitive to the initialization, like K-means. So it is required to carefully arrange the initial setup. In fact, there are mainly two approaches for the initialization. One is to apply "trial-and-error" approach, that is to randomly select single or multiple nodes for each subgraph, and calculate the value $\sum_{i=1}^{K}\mathbf{x}_i^T\mathbf{D}^{-1}\mathbf{W}\mathbf{x}_i$ when convergence reached. We must repeat these operations for several times and finally select the result with maximum amplitude as the optimal clustering. The main obstacle for this approach is the time complexity, which is $O(stN^2)$ for dense graph, and $O(stM)$ for sparse graph, where s is the repetitive times. Another more efficient approach is similar with K-means++ [34], [35], that is to select K specific nodes as the initial seeds for the K clusters. First, we choose the first node (always nodes with higher degrees to increase the success rate), then find the second node with the largest distance from the first node by some methods (such as Dijkstra algorithm). Repeat this operation to obtain the whole K seeds. Its total time complexity is $O(KN^2)$ for dense graph; while for the sparse graph, this time complexity of initialization can be decreased to $O(KM + KN\log N)$. Combined with the time cost in the main loop, the total time complexity of our algorithm is about $O(KN^2 + tN^2)$ for dense graph, and $O(KM + KN\log N + 2tM)$ for sparse graph, which is still much lower than those of its competitors.

The "K-Graph-Centers" algorithm is grounded on two novel concepts: graph cohesiveness and graph center. Graph cohesiveness quantifies the intrinsic connectivity within a graph, while graph center — represented as a nonnegative vector encoding its inverse distances to all graph vertices — serves to locate the entire graph structure. Referring to K-means, we propose the "K-Graph-Centers" algorithm. Its center updating phase (implemented via diffusion) and discretizing phase are similar with those of K-means. However, an additional normalization operation is incorporated to prevent eigenvector amplitude divergence during iteration.

"K-Graph-Centers" algorithm is rather simple and intuitive. Lying on the two proposed concepts, it allows us to transfer the K-means paradigm from common Euclidean space to the graph space. The differences are, here the graph center is not a real geometric point but a virtual entity, and its location is also not absolute coordinates but relative contributions. So "virtual graph center" describes it more precisely. Elements in the graph center vector quantify the affinities of corresponding vertices in the graph. Moreover, its derivation is not direct but in an iterative way. For mixture graph clustering, a subgraph center matrix is generated, whose spareness and effectiveness caused by the discretization, provides great convenience for this algorithm. Through combining discretization operation upon the subgraph center matrix with rapid affinity diffusion during the

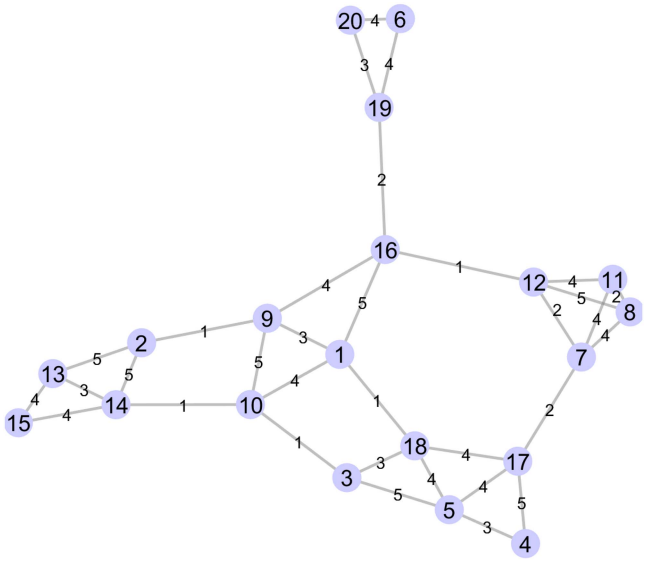
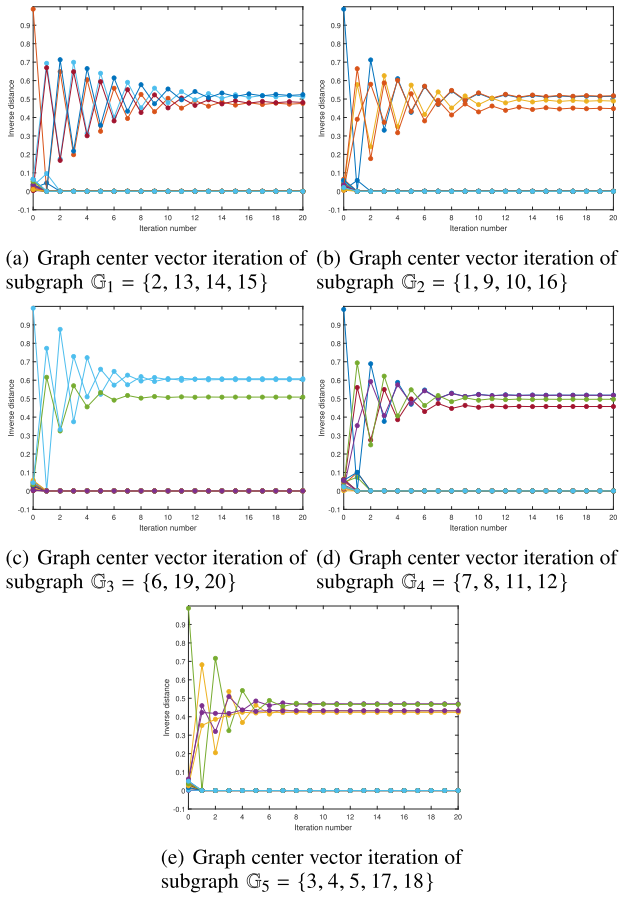


Fig. 1. Mixture graph used in feasibility test. Five clusters compose the mixture graph, with weak connections among them.



(a) Graph center vector iteration of subgraph $\mathbb{G}_1 = \{2, 13, 14, 15\}$

(b) Graph center vector iteration of subgraph $\mathbb{G}_2 = \{1, 9, 10, 16\}$

(c) Graph center vector iteration of subgraph $\mathbb{G}_3 = \{6, 19, 20\}$

(d) Graph center vector iteration of subgraph $\mathbb{G}_4 = \{7, 8, 11, 12\}$

(e) Graph center vector iteration of subgraph $\mathbb{G}_5 = \{3, 4, 5, 17, 18\}$

Fig. 2. The five subgraph centers during the iteration in "K-Graph-Centers". We can observe that all these vectors converge rapidly. Only vertices belonging to the corresponding cluster are assigned with positive inverse distance values, while the remaining ones are with zero elements.

vector updating phase together, the time efficiency of "K-Graph-Centers" is significantly improved when compared with current competitors who depend on eigen decomposition.

This algorithm introduces an innovative continuous-discretization-coupled mechanism. It circumvents the irreversible information loss inherent in the decoupled two-stage paradigm that commonly employed by conventional graph clustering methods. Though "K-Graph-Centers" still struggles against the binarization effect induced by discretization and the uniformity effect originated from affinity diffusion, the resulting subgraph centers preserve discriminative pattern in their non-zero elements, thereby extending far beyond the mere functionality of an indicator matrix.

III. Experiments on Clustering

In this section, we conduct several experiments to exhibit the algorithm comprehensively, aiming to verify the feasibility and evaluate the performance. Firstly, a feasibility test is arranged on a small graph only containing 20 nodes. The details of this algorithm are shown, such as obtained graph centers and clustering implementation. Then two synthetic datasets are adopted to examine the proposed algorithm. Finally, 10 real-world datasets of different types, dimensions and sample sizes are employed, other 10 classical graph clustering algorithms and 3 modified GNN-based methods are chosen to act as the competitors of "K-Graph-Centers". All these tests are run by MATLAB in Windows 10 with the CPU of Intel(R) Core(TM) i7-12700H@2.30 GHz.

A. Feasibility Test

Here we design a small-size graph with 20 vertices but containing 5 clusters, shown as Fig. 1. Obviously, $\{2, 13, 14, 15\}$, $\{1, 9, 10, 16\}$, $\{6, 19, 20\}$, $\{7, 8, 11, 12\}$, and $\{3, 4, 5, 17, 18\}$, are the five clusters. We apply the "K-Graph-Centers" to decompose this mixture graph. To demonstrate the convergence procedure and final subgraph centers, we monitor all the subgraph centers during the iteration, shown in Fig. 2. From these results, we can see this algorithm converges fast, typically requiring only about 10 iterations. What impresses us is that the final subgraph centers (excluding the zero elements) refuse to converge into a uniform vector. This result indicates that in the mixture graph, the subgraph centers calculated by "K-Graph-Centers" are distinguishable, thereby equipped with practical meanings. The reason for getting rid of such a curse lies on the discretization operation, which stops the affinity spreading, and maintains the distinguishability inside the subgraph by balancing with neighbor clusters. The clustering implementation is achieved by scanning row vectors of assemble subgraph center matrix $\mathbf{X}$ as mentioned in Section II-B, whose clustering result is shown in Fig. 3. We can conclude the clustering result is perfect.

B. Experiments on Synthetic Datasets

In this section, we design two synthetic datasets to test our algorithm under ideal conditions. The scatters exhibit "T-shape" pattern in the first dataset, which is composed by two uniformly distributed belts, being orthogonal with each other and each containing 200 points, as shown in Fig. 4(a); the other one is

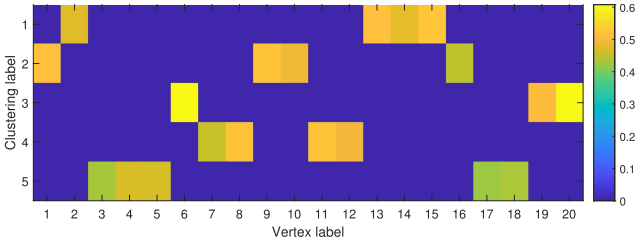
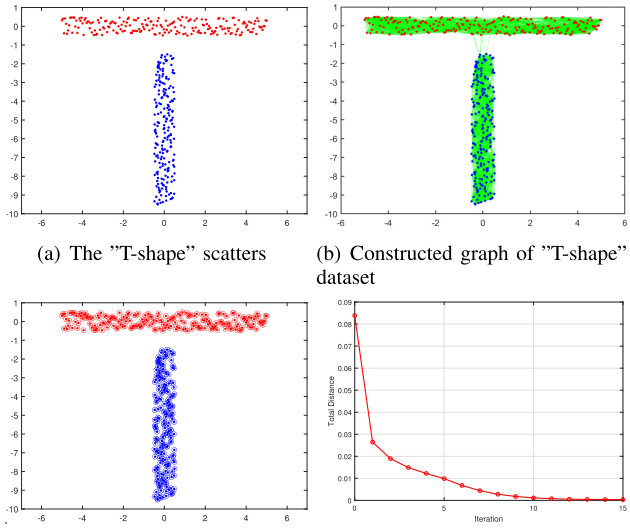


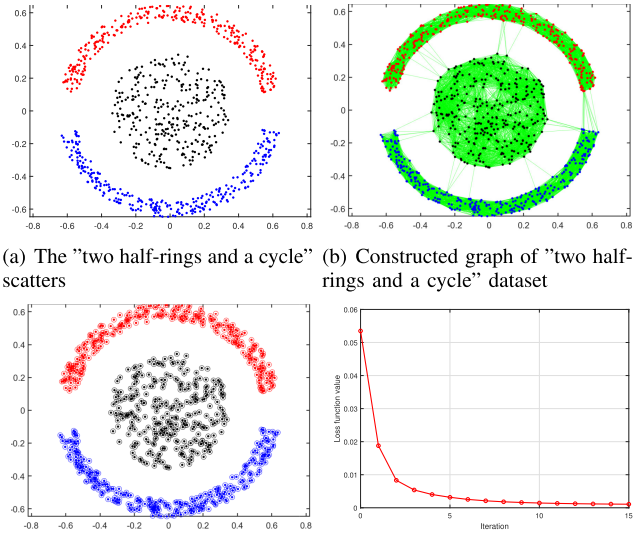
Fig. 3. Assemble subgraph center matrix $\mathbf{X}$ obtained by "K-Graph-Centers" in feasibility test. The clustering label is in vertical axis, the horizontal axis represents the vertex label. Brightness of the blocks indicates the closeness of vertices to the subgraph centers. Since the subgraph center vectors are mutually excluded, we can easily find that vertices $\{2, 13, 14, 15\}$ form the first subgraph $\mathbb{G}_1$, $\{1, 9, 10, 16\}$ are from the second subgraph $\mathbb{G}_2$, $\{6, 19, 20\}$ belong to the third subgraph $\mathbb{G}_3$, $\{7, 8, 11, 12\}$ and $\{3, 4, 5, 17, 18\}$ compose the rest two subgraphs $\mathbb{G}_4$ and $\mathbb{G}_5$.



(a) The "T-shape" scatters (b) Constructed graph of "T-shape" dataset

(c) Clustering result for "T-shape" dataset by "K-Graph-Centers" (d) Loss function curve during the iteration for "T-shape" dataset

Fig. 4. Clustering for "T-shape" dataset by "K-Graph-Centers" algorithm. From (c), we can see that the clustering is perfect, where all solid points are rounded by the outer circles of the same color, thereby indicating totally correct classifications. The curve of loss function values shown in (d) implies a fast convergence speed.



(a) The "two half-rings and a cycle" scatters (b) Constructed graph of "two half-rings and a cycle" dataset

(c) Clustering result for "two half-rings and a cycle" dataset by "K-Graph-Centers" (d) Loss function curve during the iteration for "two half-rings and a cycle" dataset

Fig. 5. Clustering for "two half-rings and a cycle" dataset by "K-Graph-Centers" algorithm. From (c), we can see that there is no error occurring in the final clustering, where all solid points and their outer rounded circles are painted by the same color, indicating completely correct classifications. The curve of loss function values is shown in (d), implying a rapid convergence in our algorithm.

TABLE I
CLUSTERING ACCURACY RESULTS AT DIFFERENT NUMBERS OF NEIGHBORS ON "T-SHAPE" DATASET

neighbor number	60	80	100	120
K-Graph-Centers	1.0000	1.0000	1.0000	0.9700
NCUT	1.0000	1.0000	0.9825	0.9600
RCUT	1.0000	1.0000	0.9850	0.9600

TABLE II
CLUSTERING ACCURACY RESULTS AT DIFFERENT NUMBERS OF NEIGHBORS ON "TWO HALF-RINGS AND A CYCLE" DATASET

neighbor number	100	120	140	160
K-Graph-Centers	1.0000	1.0000	1.0000	1.0000
NCUT	1.0000	1.0000	0.9533	0.7822
RCUT	1.0000	1.0000	0.9244	0.7467

composed by three clusters, structured by two half-rings and a circle, each of them includes 300 points, as shown in Fig. 5(a).

To construct the graph, we employ a Gaussian function as the similarity measure for samples in the datasets. Graph edges are established using the "K Nearest Neighbors (KNN)" method, which identifies a predefined number of nearest neighbors, computes the corresponding edge weights by using this expression $w(i, j) = e^{-(\mathbf{z}_i - \mathbf{z}_j)^2}$ (where $\mathbf{z}_i$ and $\mathbf{z}_j$ are the locations of node i and j), and treats all other pairs as disconnected. Here we select the numbers of neighbors in these two datasets to be 55 and 70, respectively. The resulting graphs for both datasets are visualized in Figs. 4(b) and 5(b). We then perform clustering on these graphs using the "K-Graph-Centers" algorithm, with the clustering results shown in Figs. 4(c) and 5(c). The algorithm demonstrates excellent performance on both datasets, achieving 100% accuracy. Figs. 4(d) and 5(d) record the loss function (shown in (12)) values for these datasets during the iteration. They both converge rapidly, with their loss function values decreasing nearly to zero.

The number of neighbors used in constructing the graph would affect the clustering results. We conduct the test on these two synthetic datasets by tuning the numbers of neighbors, and apply NCUT and RCUT as the competitors of our algorithm. The neighbor numbers in KNN approach for the first dataset are modified from 60, 80, 100, to 120, while for the second dataset are from 100, 120, 140 to 160. The mean clustering accuracy of 20 repetitive tests is recorded to evaluate their sensitivities about this parameter. The results are shown in Tables I and II. Applying

TABLE III
CLUSTERING ACCURACY RESULT AND TIME COST(UNIT:S) AT DIFFERENT STOP THRESHOLD ϵ ON "T-SHAPE" DATASET

ϵ	0.05	0.01	0.005	0.001
Accuracy	0.9725	0.9775	0.9800	0.9800
Time cost	0.006	0.007	0.008	0.010

TABLE IV
CLUSTERING ACCURACY RESULT AND TIME COST(UNIT:S) AT DIFFERENT STOP THRESHOLD ϵ ON "TWO HALF-RINGS AND A CYCLE" DATASET

ϵ	0.05	0.01	0.005	0.001
Accuracy	0.9878	1.0000	1.0000	1.0000
Time cost	0.005	0.010	0.015	0.022

TABLE V
PARAMETERS OF THE DATASETS USED IN THE TEST

Dataset	sample size	dimension	class
MEDMNIST	1440	784	6
SEEDS	210	7	3
COIL20	1440	1024	20
BREASTMNIST	780	784	2
FASHION-MNIST	2400	784	10
VEHICLE	846	18	4
CIFAR10	1000	3072	10
DERMATOLOGY	366	34	6
FLICKR	10708	64	7
OGB	11556	33	40

more neighborhood samples in building the graph obviously increases the graph complexity thus lowers the clustering quality. While from these tables, we can conclude that our algorithm shows higher insensitivity on the setup of neighbor number compared with NCUT and RCUT.

The stop threshold ϵ also brings effects on the clustering performance. Generally, smaller ϵ results in better clustering quality but higher time cost. While as shown the tests in Section III-A and III-B, due to the fast convergence speed of our algorithm and the sharp decrease of the loss function in the earlier phase during the iteration, K-Graph-Centers would present high robustness to this parameter. To quantitatively estimate the effect of ϵ, we design two tests on these two synthetic datasets by tuning its value from 0.05, 0.01, 0.005 to 0.001, then record the time cost and clustering accuracy. The neighbor numbers to build the graphs for these datasets are 120 and 160, respectively. The mean results of 20 repetitive tests are listed in Tables III and IV. They coincide the intuitions and imply that this algorithm shows low sensitivity to the parameter ϵ. As a result, we always set ϵ within 10^{-2} to 10^{-3} empirically.

C. Experiments on Real-World Datasets

In this section, we conduct comparative experiments on 10 commonly-used real-world datasets, which demonstrate diversities in sample size, dimension, and number of classes, aiming to comprehensively evaluate clustering capabilities. The key parameters of these datasets are listed in Table V. These datasets contains: "MEDMNIST" [36], "SEEDS" [37], "COIL20" [38], "BREASTMNIST" [39], "FASHION-MNIST" [40], "VEHICLE" [41], "CIFAR10" [42], "DERMATOLOGY" [43], "FLICKR" [44] and "OGB" [45]. For computational efficiency, random sub-sampling is applied to large-scale datasets before evaluation.

We compare the proposed approach against 10 well-established classical graph clustering algorithms, all reported to exhibit excellent performance, including: CAN [25], PCAN [25], CLR [26], RCUT [19], NCUT [20], DOGC [30], DSC [31], MCL [46], PIC [47], and Graph-embedded Subspace Clustering with Entropy-based Feature Weighting (GSCEFW) [48]. Among them, RCUT and NCUT represent foundational spectral-based clustering. RCUT computes K eigenvectors of Laplacian matrix $\mathbf{L} = \mathbf{D} - \mathbf{W}$, while NCUT employs the normalized Laplacian $\mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$. CAN derives an adaptive similarity matrix via optimization with a rank constraint, subsequently used for clustering as RCUT and NCUT do. PCAN extends CAN by incorporating an optimal projection direction learned during the iteration. CLR focuses on adaptively learning the optimal similarity matrix. DOGC dynamically constructs an optimized graph structure under a low-rank constraint, enabling direct extraction of discrete cluster assignments. DSC explicitly seeks an ideal graph topology characterized by exactly K mutually disconnected components to implement clustering directly. GSCEFW tries to obtain the optimal graph guided by a pseudo-label learning and reduce the bad effect from noisy or irrelevant features by introducing an entropy-based feature weighting term. Notably, all compared algorithms except RCUT and NCUT follow a two-stage iterative paradigm: they alternate between updating the Laplacian (or similarity) matrix and an indicator matrix, before performing the final clustering using either the converged similarity matrix or the refined indicator matrix. Moreover, all these 8 competitors require to compute the first K eigenvectors of the Laplacian matrix or conduct the equivalent calculation for one time (RCUT and NCUT) or multiple times (CAN, PCAN, CLR, DOGC, DSC and GSCEFW). While Markov Clustering (MCL) simulates random walks on a graph and alternates expansion with inflation to strengthen dense intra-cluster flows and eliminate weak inter-cluster ones, eventually converging to a hard clustering. Power Iteration Clustering (PIC) computes the dominant eigenvector of the normalized adjacency matrix via power iteration and then extracts clusters by applying the clustering operation on the obtained eigenvector. All algorithms were initialized as specified in their original publications here. Additionally, besides these classical graph clustering methods, emerging GNN-based approaches cannot be neglected. With some modifications, they can also be adapted to graph clustering tasks. Here we employ three recent GNN-based methods and modify them to be qualified as our competitors, aiming to comprehensively evaluate the performance of our algorithm: Graph Linear Convolution Pooling Network (GLCPN) [49], graph Conjoint ATtention networks (CATs) [50], and Polarized Message-Passing Graph Attention Network (PMP-GAT) [51]. Specifically, GLCPN leverages simplified graph convolutions to capture high-order connectivity for learning matrix factorization representations, adopts a priori convolution operator in each layer for node-node collaboration aggregation, and finally uses a locality-enhanced pooling scheme to holistically utilize multi-layer neighborhood representations. CATs incorporates

TABLE VI
ACCURACY (ACC) RESULTS ON THESE DATASETS FOR THE CLUSTERING ALGORITHMS

Method	MEDMNIST	SEEDS	COIL20	BREASTMNIST	FASHION-MNIST	VEHICLE	CIFAR10	DERMATOLOGY	FLICKR	OGB
CAN	**1.0000(0.0000)**	0.7476(0.0000)	**0.8458(0.0000)**	0.7256(0.0000)	0.8467(0.0000)	**0.7329(0.0000)**	0.7220(0.0000)	0.3994(0.0000)	0.9018(0.0000)	0.5917(0.0000)
PCAN	0.9958(0.0000)	0.7476(0.0000)	0.7125(0.0000)	0.9305(0.0000)	0.8450(0.0000)	0.7328(0.0000)	0.5860(0.0000)	0.5140(0.0000)	0.9684(0.0000)	0.6099(0.0000)
CLR	0.9944(0.0000)	0.6429(0.0000)	0.6861(0.0000)	0.9308(0.0000)	0.4850(0.0000)	0.7317(0.0000)	0.3360(0.0000)	0.4972(0.0000)	**0.9711(0.0000)**	0.6094(0.0000)
RCUT	0.9892(0.0017)	0.7967(0.0109)	0.7755(0.0005)	0.7564(0.0000)	0.8371(0.0020)	0.6580(0.0043)	0.4624(0.0012)	0.4201(0.0015)	0.9657(0.0003)	0.7501(0.0017)
NCUT	0.9983(0.0000)	0.8905(0.0000)	0.7699(0.0003)	0.7885(0.0000)	0.8327(0.0000)	0.6649(0.0000)	0.7834(0.0009)	0.3957(0.0001)	0.9709(0.0000)	0.7692(0.0010)
DOGC	0.3473(0.0016)	0.7335(0.0066)	0.1901(0.0003)	0.5827(0.0266)	0.7291(0.0043)	0.4857(0.0065)	0.1627(0.0001)	0.6004(0.0067)	0.7598(0.0034)	0.1761(0.0040)
DSC	0.9986(0.0000)	0.8850(0.0000)	0.7848(0.0001)	0.7949(0.0000)	**0.8884(0.0003)**	0.6687(0.0001)	0.4525(0.0007)	0.3835(0.0003)	0.9709(0.0000)	0.7397(0.0019)
MCL	0.9417(0.0000)	**0.9143(0.0000)**	0.5757(0.0000)	0.5872(0.0000)	0.7104(0.0000)	0.4326(0.0000)	0.6870(0.0000)	0.4274(0.0000)	0.9326(0.0000)	0.4312(0.0000)
PIC	0.9984(0.0003)	0.7155(0.0339)	0.8010(0.0225)	0.9380(0.0003)	0.6993(0.0414)	0.4746(0.0652)	0.4220(0.0093)	0.3242(0.0246)	0.9705(0.0004)	0.7608(0.0478)
CATs	0.7765(0.1295)	0.8019(0.0065)	0.6457(0.0298)	0.8826(0.0769)	0.8055(0.0938)	0.6908(0.0185)	0.4498(0.1276)	0.3690(0.0792)	0.9688(0.0002)	0.8295(0.0164)
GLCPN	0.9771(0.0356)	0.8226(0.0718)	0.7549(0.0239)	0.8735(0.0828)	0.8757(0.0403)	0.6966(0.0468)	0.6884(0.0508)	0.4263(0.0371)	0.9446(0.0492)	0.8320(0.0183)
PMP-GAT	0.8835(0.1488)	0.6187(0.2330)	0.6286(0.0136)	0.9305(0.0072)	0.7797(0.2637)	0.7095(0.0888)	0.5629(0.0394)	0.6168(0.2023)	0.9676(0.0005)	0.7090(0.1321)
GSCEFW	0.9979(0.0000)	0.8906(0.0007)	0.7354(0.0137)	0.8038(0.0000)	0.8842(0.0002)	0.6265(0.0000)	**0.7872(0.0045)**	**0.8967(0.0008)**	0.9668(0.0000)	0.1425(0.0000)
K-Graph-Centers	0.9985(0.0003)	0.8997(0.0013)	0.8437(0.0024)	**0.9384(0.0050)**	0.8843(0.0007)	0.6923(0.0461)	0.6791(0.0127)	0.4978(0.0243)	0.9684(0.0000)	**0.8390(0.0000)**

TABLE VII
NORMALIZED MUTUAL INFORMATION (NMI) RESULTS ON THESE DATASETS FOR THE CLUSTERING ALGORITHMS

Method	MEDMNIST	SEEDS	COIL20	BREASTMNIST	FASHION-MNIST	VEHICLE	CIFAR10	DERMATOLOGY	FLICKR	OGB
CAN	**1.0000(0.0000)**	0.5321(0.0000)	**0.9200(0.0000)**	0.5118(0.0000)	0.9008(0.0000)	0.7042(0.0000)	0.6350(0.0000)	0.3612(0.0000)	0.7693(0.0000)	0.7850(0.0000)
PCAN	0.9879(0.0000)	0.5517(0.0000)	0.8081(0.0000)	0.6045(0.0000)	0.8690(0.0000)	0.7006(0.0000)	0.5127(0.0000)	0.6755(0.0000)	0.8933(0.0000)	0.8156(0.0000)
CLR	0.9835(0.0000)	0.4295(0.0000)	0.8160(0.0000)	0.5819(0.0000)	**0.9032(0.0000)**	0.7031(0.0000)	0.3578(0.0000)	0.4612(0.0000)	0.8848(0.0000)	0.8127(0.0000)
RCUT	0.9913(0.0003)	0.6169(0.0080)	0.8641(0.0001)	0.4780(0.0001)	0.8848(0.0002)	0.6369(0.0076)	0.5575(0.0007)	0.3729(0.0012)	0.8940(0.0003)	0.8690(0.0002)
NCUT	0.9943(0.0000)	0.7097(0.0000)	0.8828(0.0001)	0.4932(0.0001)	0.8873(0.0000)	0.7405(0.0000)	**0.7005(0.0002)**	0.3620(0.0001)	0.8995(0.0000)	0.8735(0.0001)
DOGC	0.1429(0.0014)	0.5433(0.0194)	0.1528(0.0002)	0.4338(0.0012)	0.7428(0.0027)	0.2492(0.0127)	0.0309(0.0000)	0.5414(0.0135)	0.7143(0.0013)	0.1572(0.0053)
DSC	0.9955(0.0000)	0.7158(0.0000)	0.8764(0.0000)	0.5096(0.0000)	0.9015(0.0000)	**0.7428(0.0001)**	0.5740(0.0003)	0.3619(0.0003)	**0.8995(0.0000)**	0.8667(0.0002)
MCL	0.9394(0.0000)	0.7133(0.0000)	0.8074(0.0000)	0.2402(0.0000)	0.7549(0.0000)	0.4335(0.0000)	0.6328(0.0000)	0.4355(0.0000)	0.7913(0.0000)	0.7408(0.0000)
PIC	0.7859(0.0783)	0.1583(0.0382)	0.8693(0.0087)	**0.6158(0.0015)**	0.7403(0.0471)	0.2868(0.1035)	0.4728(0.0049)	0.3172(0.0574)	0.8906(0.0008)	0.8694(0.0131)
CATs	0.8153(0.1013)	0.6922(0.0185)	0.7169(0.0164)	0.4478(0.2270)	0.8127(0.0555)	0.7191(0.0210)	0.4554(0.1083)	0.1982(0.1334)	0.8889(0.0007)	0.8731(0.0116)
GLCPN	0.9570(0.0326)	0.5898(0.0852)	0.8110(0.0139)	0.4829(0.1414)	0.8348(0.0235)	0.5844(0.0617)	0.6094(0.0305)	0.2825(0.0536)	0.8650(0.0322)	0.8883(0.0092)
PMP-GAT	0.8557(0.1595)	0.3332(0.2115)	0.6624(0.0108)	0.5705(0.0338)	0.8045(0.0492)	0.6209(0.1036)	0.4629(0.0425)	0.5306(0.1488)	0.8829(0.0018)	0.7645(0.0904)
GSCEFW	0.9930(0.0000)	0.6693(0.0013)	0.8288(0.0027)	0.3167(0.0000)	0.8488(0.0001)	0.6495(0.0000)	0.6951(0.0019)	**0.8086(0.0034)**	0.8767(0.0000)	0.2202(0.0000)
K-Graph-Centers	0.9956(0.0010)	**0.7227(0.0049)**	0.8844(0.0028)	0.6157(0.0174)	0.8802(0.0006)	0.5779(0.1092)	0.6351(0.0086)	0.3852(0.0159)	0.8822(0.0000)	**0.8978(0.0000)**

TABLE VIII
PURITY (PUR) RESULTS ON THESE DATASETS FOR THE CLUSTERING ALGORITHMS

Method	MEDMNIST	SEEDS	COIL20	BREASTMNIST	FASHION-MNIST	VEHICLE	CIFAR10	DERMATOLOGY	FLICKR	OGB
CAN	**1.0000(0.0000)**	0.7476(0.0000)	**0.9458(0.0000)**	0.7256(0.0000)	0.8854(0.0000)	**0.9811(0.0000)**	0.7920(0.0000)	0.5670(0.0000)	**0.9726(0.0000)**	**0.8403(0.0000)**
PCAN	0.9958(0.0000)	0.7476(0.0000)	0.8090(0.0000)	0.9359(0.0000)	0.8846(0.0000)	0.9787(0.0000)	0.7070(0.0000)	0.7793(0.0000)	0.9684(0.0000)	0.7964(0.0000)
CLR	0.9944(0.0000)	0.8143(0.0000)	0.9146(0.0000)	0.9308(0.0000)	0.8779(0.0000)	0.9799(0.0000)	0.7760(0.0000)	0.5559(0.0000)	0.9711(0.0000)	0.6834(0.0000)
RCUT	0.9906(0.0012)	0.7983(0.0104)	0.8106(0.0003)	0.7564(0.0000)	0.8674(0.0010)	0.6892(0.0030)	0.4654(0.0012)	0.5539(0.0007)	0.9657(0.0003)	0.7686(0.0014)
NCUT	0.9983(0.0000)	0.8905(0.0000)	0.8149(0.0002)	0.7885(0.0000)	0.8807(0.0000)	0.7376(0.0000)	0.7852(0.0006)	0.5384(0.0000)	0.9709(0.0000)	0.7846(0.0007)
DOGC	0.3659(0.0013)	0.7351(0.0060)	0.2176(0.0003)	0.7308(0.0000)	0.7346(0.0040)	0.4994(0.0062)	0.1699(0.0001)	0.6735(0.0066)	0.8651(0.0014)	0.1763(0.0040)
DSC	0.9986(0.0000)	0.8850(0.0000)	0.8181(0.0001)	0.7949(0.0000)	0.8886(0.0003)	0.7376(0.0000)	0.4556(0.0007)	0.5365(0.0001)	0.9709(0.0000)	0.7595(0.0014)
MCL	0.9417(0.0000)	**0.9143(0.0000)**	0.6729(0.0000)	0.5872(0.0000)	0.7104(0.0000)	0.4326(0.0000)	**0.8350(0.0000)**	0.4330(0.0000)	0.9326(0.0000)	0.4312(0.0000)
PIC	0.9984(0.0003)	0.7366(0.0028)	0.8255(0.0163)	0.9380(0.0003)	0.7521(0.0594)	0.5014(0.0629)	0.4532(0.0016)	0.3911(0.0368)	0.9705(0.0004)	0.7792(0.0427)
CATs	0.7765(0.1295)	0.9019(0.0065)	0.6481(0.0310)	0.8826(0.0769)	0.8055(0.0938)	0.7504(0.0181)	0.4499(0.1276)	0.3807(0.0806)	0.9688(0.0002)	0.8095(0.0164)
GLCPN	0.9771(0.0356)	0.8232(0.0700)	0.7616(0.0204)	0.8794(0.0682)	0.8777(0.0353)	0.6986(0.0442)	0.6963(0.0528)	0.4990(0.0356)	0.9543(0.0236)	0.8322(0.0182)
PMP-GAT	0.9610(0.0366)	0.8402(0.1042)	0.6490(0.0126)	0.9305(0.0072)	0.8814(0.0237)	0.8417(0.0849)	0.6186(0.0444)	0.7441(0.0548)	0.9700(0.0024)	0.7154(0.1289)
GSCEFW	0.9979(0.0000)	0.8906(0.0007)	0.7798(0.0095)	0.8038(0.0000)	0.8847(0.0002)	0.7293(0.0000)	0.7872(0.0045)	**0.8967(0.0008)**	0.9668(0.0000)	0.1553(0.0000)
K-Graph-Centers	0.9985(0.0003)	0.8997(0.0013)	0.8635(0.0040)	**0.9384(0.0050)**	**0.9337(0.0007)**	0.7678(0.0724)	0.7749(0.0150)	0.5297(0.0285)	0.9684(0.0000)	0.8390(0.0000)

heterogeneous learnable factors (internal and/or external to the neural network) to compute more purposeful and appropriate attention coefficients. PMP-GAT combines a polarized message-passing paradigm with a graph attention mechanism, where the polarized message-passing captures both node similarity and dissimilarity to acquire dual sources of messages from neighbors. Under the maintenance of core ideas and modules in these GNN-based methods, the modifications mainly focus on decreasing the network layers, setting the input graph signal into one-hot indicator, initializing several samples as diffusion seeds, removing the parallel computation module, and running only forward propagation or multiple power iterations, and so on, to adapt to the unsupervised/semi-supervised graph clustering task.

The adjacency matrices for graph construction across all datasets are generated also by using a Gaussian kernel function combined with a KNN approach. Algorithm performance is evaluated upon four metrics: clustering accuracy (ACC), Normalized Mutual Information (NMI), Purity (PUR), and computational cost. For robustness, all experiments have been independently repeated 100 times. We report both the mean value and standard deviation for each metric. Their results are recorded in Tables VI, VII, VIII, and IX, where bold indicates the optimal performance, and underline means the second optimal one.

From the metrics recorded in these tables, we can conclude that our algorithm "K-Graph-Centers" works very well on all the datasets when compared with other competitors, especially on the computational cost metric. For ACC, NMI and PUR three metrics, if counting the optimal and sub-optimal values for all the algorithms, we find that "K-Graph-Centers", CAN, and DSC are the top three candidates. They possess 14, 11, and 8 hits respectively on these metrics. Among them, "K-Graph-Centers" features the best performance, which verifies the high effectiveness of its core ideas. Three simple and intuitive operations — sparse matrix multiplication, row vector discretization, and column vector normalization — collectively cause this impressive performance. The first operation realizes the assemble subgraph center matrix updating, which basically represents one-step affinity diffusion for current nodes in the cluster, then the following discretization can be viewed as a competition for target nodes among these clusters, with a

TABLE IX
COMPUTATIONAL COST RESULTS ON THESE DATASETS FOR THE CLUSTERING ALGORITHMS(UNIT:S)

Method	MEDMNIST	SEEDS	COIL20	BREASTMNIST	FASHION-MNIST	VEHICLE	CIFAR10	DERMATOLOGY	FLICKR	OGB
CAN	0.78(0.04)	0.03(0.01)	1.99(0.12)	0.10(0.05)	7.42(0.11)	0.59(0.02)	0.70(0.04)	0.07(0.01)	1626(225.0)	2178(459.7)
PCAN	2.42(0.15)	0.04(0.01)	20.19(0.32)	0.45(0.02)	8.97(0.43)	0.65(0.03)	1.91(0.04)	0.10(0.01)	2253(244.5)	2716(211.7)
CLR	49.81(0.71)	2.56(0.05)	70.39(4.00)	13.28(0.44)	213.3(0.40)	16.64(0.24)	19.98(0.19)	4.19(0.08)	1.63×10^4(136.4)	2.32×10^4(155.5)
RCUT	0.23(0.00)	0.02(0.00)	0.18(0.00)	0.02(0.00)	0.09(0.00)	0.31(0.00)	0.15(0.00)	0.07(0.00)	134.3(22.85)	163.6(26.77)
NCUT	0.57(0.81)	0.08(0.00)	0.54(0.36)	0.14(0.12)	0.41(0.89)	0.53(0.28)	0.49(0.40)	0.23(0.23)	417.0(189.6)	466.9(196.5)
DOGC	1.75(0.01)	0.03(0.00)	1.36(1.37)	0.06(0.00)	2.48(0.00)	0.88(0.00)	1.86(0.02)	0.13(0.00)	264.3(95.83)	475.5(227.3)
DSC	9.55(1.02)	0.45(0.00)	7.06(0.54)	0.52(0.00)	5.27(0.32)	5.13(0.01)	6.68(0.53)	1.01(0.01)	1539(182.9)	1661(191.3)
MCL	3.60(0.24)	0.07(0.00)	3.88(0.22)	0.74(0.01)	14.77(0.34)	0.91(0.03)	1.41(0.05)	0.14(0.01)	509.0(46.60)	771.3(55.28)
PIC	0.21(0.04)	0.05(0.04)	0.41(0.05)	0.08(0.04)	0.99(0.14)	0.10(0.04)	0.17(0.05)	0.07(0.05)	17.56(2.12)	35.39(0.61)
CATs	40.90(2.97)	10.20(1.15)	39.00(1.66)	24.67(1.57)	49.80(2.74)	23.75(1.34)	33.33(1.40)	12.47(1.42)	1350(86.49)	1783(93.83)
GLCPN	1.36(0.06)	0.46(0.05)	1.75(0.06)	0.80(0.06)	3.44(0.12)	0.83(0.06)	1.17(0.14)	0.66(0.04)	83.33(6.76)	92.20(8.13)
PMP-GAT	6.17(0.37)	1.25(0.27)	7.10(0.35)	5.68(0.27)	8.45(0.69)	5.77(0.40)	7.22(0.56)	3.39(0.29)	286.6(29.12)	332.8(38.46)
GSCEFW	2.75(0.54)	0.16(0.06)	6.70(1.00)	1.52(0.23)	17.98(0.45)	1.48(0.03)	2.65(0.06)	0.26(0.02)	1347(55.14)	1690(60.40)
K-Graph-Centers	**0.01(0.03)**	**0.001(0.00)**	**0.04(0.01)**	**0.004(0.002)**	**0.05(0.12)**	**0.01(0.02)**	**0.02(0.05)**	**0.002(0.004)**	**1.01(0.01)**	**1.53(0.01)**

criterion of "winner-takes-all". The normalization can stop the explosion or vanishing. The final steady state represents the optimal locations of the subgraph centers. Therefore, the pattern in mixture graph has been embedded into this subgraph center matrix. The success of CAN and DSC lies in the searching for optimal similarity matrix. They both modify the original similarity matrix to achieve a most significant graph structure with highest separability, thus improve their clustering performances. PCAN originates from CAN, but an additional projection is applied to the original data, aiming to find an optimal direction which can exhibit better segmentation ability. However, the introducing of new constraint term to the loss function may bring difficulty to obtain the optimal solutions, and obviously increase the computational complexity. CLR also attempts to learn a better graph structure, but its limit comes from the term to constrain fidelity with the original similarity matrix. Thus its result relies badly on the quality of original similarity matrix. While in the DOGC method, besides the new similarity matrix, a spectral rotation matrix and a discrete indicator matrix are also introduced in the optimizing objectives, thus cause the optimizing issue more complex and difficult to obtain the best quality. More hyper-parameters require more time to tune until optimal quality reached, hindering its further applications. RCUT and NCUT show moderate and similar performances on all datasets, whose robustness shows that they are suitable to act as two baseline methods for spectral-based methods. MCL is sensitive to the parameter setup, making the clustering result hard to reach the optimum. PIC shares the similar form as our algorithm by utilizing the power iteration, but it doesn't apply nonnegative constraint on $\mathbf{x}$ or $\mathbf{X}$ and misses the discretization operation. Therefore, the critical factor hindering the application of PIC is the stop criterion, which is complicate to set but always replies on an empirical "early stop", causing the results low reproductivity. In GSCEFW, it requires to optimize up to 6 objectives, whose best solutions are really difficult to reach. For the GNN-based methods(GLCPN, CATs and PMP-GAT), though under some delicate modifications and tuning, their intrinsic structure and paradigm constrain further performance quality.

What mostly interests us is the computational time of this "K-Graph-Centers" algorithm. From Table IX, we can observe that on all datasets, it achieves the lowest computational complexity among these competitors. Though underwent fine optimization in Matlab, RCUT and NCUT still cost more time than "K-Graph-Centers", nevertheless other variants (such as CAN, PCAN, CLR, DOGC, DSC, and GSCEFW) based on them. "K-Graph-Centers" even exhibits computational costs several orders of magnitude lower than RCUT and NCUT in most datasets. And such an advantage presents more significantly as the dataset size and class number increases, promising broad industrial applications in large-scale network analysis. The reason comes from that, unlike RCUT, NCUT and their variants involve a complete and explicit eigen decomposition conducted for single time or multiple times, "K-Graph-Centers" replaces the eigen decomposition by fast diffusion and hard assignment, thus realizing direct graph clustering at the steady state. This algorithm fully exploits the sparseness induced by the discretization on the subgraph center matrix to accelerate the convergence. MCL is obviously not comparable with K-Graph-Centers in time cost, due to its power iteration completely upon the adjacency matrix, producing a computational complexity of $O(N^3)$. The time cost of PIC seems acceptable among the candidates. However, such an achievement is difficult to reproduce, and mainly comes from a complicated and time-consumed parameter setup. While for GNN-based methods, their deep-learning structures inherently consume a significant amount of time, even with delicate modifications.

From these tests in this section, we verify the feasibility of "K-Graph-Centers" in graph clustering, demonstrate its capability and superiority on synthetic datasets and real-world datasets. Generally, "K-Graph-Centers" presents excellent performance compared with other competitors, especially on the computational cost metric. This algorithm is suitable to analyze large-scale graph. The "virtual graph center" obtained from this algorithm is distinguishable, thus reveals more insights for the graph structure. Total subgraph cohesiveness also increases during the iteration, confirming the model correctness and algorithm convergence.

IV. CONCLUSION AND FORECASTING

This paper introduces a "K-Graph-Centers" algorithm to do the graph clustering. It is based on the "graph cohesiveness" and "graph center" concepts. The "graph center" is a virtual node, representing the graph affinity center. Its elements record the inverse distances from itself to vertices within the cluster, thus encodes the relative topology information for the graph. Based on the definition of graph center, graph cohesiveness

is proposed to measure the graph compactness, which quantitatively equals to the total weighted inverse distances from the vertices to their corresponding graph centers. Maximizing the graph cohesiveness relates to search certain eigenvector variant of the normalized adjacency matrix. Aiming to embed the partition into iteration and balance the effect from neighbor clusters, we add a discretization operation after the eigenvector updating. Such a combination brings great advantages: firstly, it replaces the explicit eigen decomposition by a simple diffusion, thus reducing the redundancy and complexity; the second is that the coupling of diffusion and discretization, accelerates the convergence speed and decreases the time cost; finally, the discretization maintains sparseness during the iteration, thus brings a high efficiency in the matrix multiplication. Of course a normalization operation is designed to satisfy the eigenvector amplitude constraint. At the steady state, the final subgraph centers are all mutually exclusive or orthogonal, thus clustering can be easily implemented by just examining the non-zero element location row by row. Beyond the cluster indicator information, the subgraph centers also encode the vertex affinity or importance to the corresponding cluster, providing a specific low-dimension mapping for the original graph topology.

Its similarity with K-means comes from the same scheme: the subgraph center updating of "K-Graph-Centers" can be viewed as mean calculation in K-means; while its discretization is similar with the cluster assignment in K-means. But the differences are also significant. In the first step, K-means determines its current cluster centers exactly; while in this algorithm, the subgraph centers are approximated by once iteration, which is only one-step approaching, not the final true value. Their second phases also differ: the discretization in K-means only keeps the one-hot indicators, while here the inverse distance information is also encoded.

From the theoretical analysis, "K-Graph-Centers" exhibits a time complexity of $O(tN^2)$ on dense graphs. This achievement is rather encouraging, since other competitors often contain once or multiple complete eigen decomposition, thus with a computational complexity of $O(N^3)$ or $O(tcKN^2)$. When mixture graph is sparse, the time efficiency advantage will emerge more significantly in "K-Graph-Centers", with a complexity of only $O(t|\mathbb{E}|)$, which is linear with edge number.

The subgraph center updating can be considered as affinity diffusion within the cluster, while the following discretization behaves as a competition among all clusters for each graph vertex. Discretization also stops the affinity diffusing to all vertices uniformly within the subgraphs, ensuring subgraph center matrix meaningful and distinguishable. This coupling of these two operations accelerates the convergence, and avoids the information loss, which always occurs in the final step of other spectral-based methods.

The experiments conducted in this paper demonstrate the superiority of "K-Graph-Centers" algorithm. In the feasibility test, it can be observed that its clustering is perfect, the convergence is fast, and the subgraph centers are meaningful and distinct. In the tests on two synthetic datasets, "K-Graph-Centers" performs excellently, whose loss function decreases rapidly. The parameter sensitivity tests on neighbor number and stop threshold indicate that our algorithm also presents good robustness on these two parameters. The last experiment on 10 real-world datasets demonstrates that "K-Graph-Centers" can supply comparable, sometimes even the best performance in most metrics against other classical or state-of-art competitors. Crucially, across all the datasets, its time cost is the lowest, specifically, commonly 10^{-1} to 10^{-2} smaller than the best of remaining methods. This result coincides with our theoretical analysis, and such a superiority will be more remarkable on large-scale datasets. Notably that this efficiency achievement is realized just on a raw implementation version of "K-Graph-Centers", while most spectral-based competitors are highly optimized in eigen decomposition by Matlab.

"K-Graph-Centers" breaks the greatest obstacle that hinders further application of spectral-based clustering methods for industrial large-scale datasets by providing a linear computational complexity. It proposes an implicit spectral clustering paradigm: to replace the complicate eigen decomposition and irreversible discretization by affinity diffusion and contribution sparsification. The alternate continuous-discrete-coupled optimization maintains the intrinsic information and promises a rapid convergence, which can inspire us to extend it into similar fields. The subgraph centers quantify node cohesiveness contribution to the subgraphs, leading to differentiable and dynamic graph analysis. Graph cohesiveness provides a quantitative measure of graph features that enables manipulation mathematically. These two concepts have the potential to serve as powerful tools for further graph research.

The drawbacks of "K-Graph-Centers" are also not negligible. Firstly, it is intrinsically sensitive to the initialization as K-means, wrong initial setup will lead to local optimum. Possible solutions can also refer to K-means, such as trial-and-error, or tricks as K-means++ and initKmix [52]. The second one is the hard partition induced by discretization, which brings difficulty in processing the overlapping community. Some fuzzy approaches can be introduced to address this issue [53], [54].

REFERENCES

[1] A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of Big Data," *Inf. Sci.*, vol. 622, pp. 178–210, 2023.

[2] P. Zhou, L. Du, and X. Li, "Adaptive consensus clustering for multiple K-means via base results refining," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10251–10264, Oct. 2023.

[3] J. Wang et al., "Fast approximated multiple kernel K-means," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6171–6180, Nov. 2024.

[4] D. Li, S. Zhou, T. Zeng, and R. H. Chan, "Multi-prototypes convex merging based K-means clustering algorithm," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6653–6666, Nov. 2024.

[5] M. Zaghloul, S. Barakat, and A. Rezk, "Enhancing customer retention in online retail through churn prediction: A hybrid RFM, K-means, and deep neural network approach," *Expert Syst. Appl.*, vol. 290, 2025, Art. no. 128465.

[6] R. Wei, Y. Liu, J. Song, Y. Xie, and K. Zhou, "Exploring hierarchical information in hyperbolic space for self-supervised image hashing," *IEEE Trans. Image Process.*, vol. 33, pp. 1768–1781, 2024.

[7] M. Hamad, C. Conti, P. Nunes, and L. D. Soares, "Hyperpixels: Flexible 4 d over-segmentation for dense and sparse light fields," *IEEE Trans. Image Process.*, vol. 32, pp. 3790–3805, 2023.

[8] X.-Y. Shih, H.-C. Chen, and X.-L. Hung, "Design and hardware implementation of low-latency 4-splitting tree-structure-based K-means clustering

trainer and classifier chip for detecting cybersecurity attacks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 4, pp. 613–617, Apr. 2025.
[9] A. Aftiss, S. Lamsiyah, S. Ouatik El Alaoui, and C. Schommer, "BioMD-Sum: An effective hybrid biomedicalmulti-document summarization method based on pagerank and longformer Encoder-Decoder," *IEEE Access*, vol. 12, pp. 188013–1880 31, 2024.
[10] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "Density-based spatial clustering of applications with noise," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, vol. 240, no. 6, 1996, pp. 226–231.
[11] A. Hinneburg and D. A. Keim, "An efficient approach to clustering in large multimedia databases with noise," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 1998, pp. 58–65.
[12] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Science*, vol. 344, no. 6191, pp. 1492–1496, 2014.
[13] W. Wang, J. Yang, and R. Muntz, "STING: A statistical information grid approach to spatial data mining," in *Proc. 23rd Int. Conf. Very Large Data Bases*, Athens, Greece, 1997, pp. 186–195.
[14] F. Ma, C. Wang, J. Huang, Q. Zhong, and T. Zhang, "Key grids based batch-incremental clique clustering algorithm considering cluster structure changes," *Inf. Sci.*, vol. 660, 2024, Art. no. 120109.
[15] G. Sheikholeslami, S. Chatterjee, and A. Zhang, "WaveCluster: A multi-resolution clustering approach for very large spatial databases," in *Proc. 24rd Int. Conf. Very Large Data Bases*, 1998, vol. 98, pp. 428–439.
[16] J. Yao and Y. Zeng, "Gauging-: Border-aware hierarchical clustering based on density and proximity," *Pattern Recognit.*, vol. 171, 2026, Art. no. 112128.
[17] Y. Lin, H. Hu, B. Li, S. Zhao, and H. Jing, "Representation auto-fused NMF based hierarchical clustering," *Expert Syst. Appl.*, vol. 283, 2025, Art. no. 127560.
[18] J. W. Sangma, Yogita, V. Pal, N. Kumar, and R. Kushwaha, "FHC-NDS: Fuzzy hierarchical clustering of multiple nominal data streams," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 3, pp. 786–798, Mar. 2023.
[19] L. Hagen and A. B. Kahng, "New spectral methods for ratio cut partitioning and clustering," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 11, no. 9, pp. 1074–1085, Sep. 1992.
[20] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 8, pp. 888–905, Aug. 2000.
[21] Y. Pang, J. Xie, F. Nie, and X. Li, "Spectral clustering by joint spectral embedding and spectral rotation," *IEEE Trans. Cybern.*, vol. 50, no. 1, pp. 247–258, Jan. 2020.
[22] C. Gao, W. Chen, F. Nie, W. Yu, and Z. Wang, "Spectral clustering with linear embedding: A discrete clustering method for large-scale data," *Pattern Recognit.*, vol. 151, 2024, Art. no. 110396.
[23] Y. Yang, F. Shen, Z. Huang, H. T. Shen, and X. Li, "Discrete nonnegative spectral clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 9, pp. 1834–1845, Sep. 2017.
[24] R. Wang, H. Chen, Y. Lu, Q. Zhang, F. Nie, and X. Li, "Discrete and balanced spectral clustering with scalability," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 14321–14336, Dec. 2023.
[25] F. Nie, X. Wang, and H. Huang, "Clustering and projected clustering with adaptive neighbors," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 977–986.
[26] F. Nie, X. Wang, M. Jordan, and H. Huang, "The constrained Laplacian rank algorithm for graph-based clustering," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016, pp. 1969–1976.
[27] S. Park and H. Zhao, "Spectral clustering based on learning similarity matrix," *Bioinformatics*, vol. 34, no. 12, pp. 2069–2076, 2018.
[28] Z. Kang, C. Peng, Q. Cheng, and Z. Xu, "Unified spectral clustering with optimal graph," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 3366–3373.
[29] Z. Bian, H. Ishibuchi, and S. Wang, "Joint learning of spectral clustering structure and fuzzy similarity matrix of data," *IEEE Trans. Fuzzy Syst.*, vol. 27, no. 1, pp. 31–44, Jan. 2019.
[30] Y. Han, L. Zhu, Z. Cheng, J. Li, and X. Liu, "Discrete optimal graph clustering," *IEEE Trans. Cybern.*, vol. 50, no. 4, pp. 1697–1710, Apr. 2020.
[31] F. Nie, C. Liu, R. Wang, and X. Li, "A novel and effective method to directly solve spectral clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10863–10875, Dec. 2024.
[32] J. Y. J. Z. Y. X, S Zhao, and B. Zhang, "Linear discriminant analysis," *Nature Rev. Methods Primers*, vol. 4, no. 70, pp. 1–16, 2024.
[33] J. Garza-Vargas and A. Kulkarni, "The Lanczos algorithm under few iterations: Concentration and location of the output," *SIAM J. Matrix Anal. Appl.*, vol. 41, no. 3, pp. 1312–1346, 2020.
[34] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2007, pp. 1027–1035.
[35] H. Zhang and J. Li, "Towards faster seeding for K-means via lower bound and triangle inequality," *Neurocomputing*, vol. 639, 2025, Art. no. 130227.
[36] J. Yang et al., "MedMNIST v2-A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Sci. Data*, vol. 10, no. 1, 2023, Art. no. 41.
[37] M. Charytanowicz, J. Niewczas, P. Kulczycki, P. A. Kowalski, S. Łukasik, and S. Żak, "Complete gradient clustering algorithm for features analysis of X-ray images," in *Proc. Inf. Technol. Biomed.*, Berlin, Germany: Springer, 2010, pp. 15–24.
[38] S. N. S. A. Nene and H. Murase, "Columbia object image library," (COIL-20) *Dept. Comput. Sci., Columbia Univ., New York, NY, USA, Tech. Rep.* CUCS-005-96, Feb. 1996.
[39] J. Yang, R. Shi, and B. Ni, "MedMNIST classification decathlon: A lightweight automl benchmark for medical image analysis," in *Proc. IEEE 18th Int. Symp. Biomed. Imag.*, 2021, pp. 191–195.
[40] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017.
[41] P. Mowforth and B. Shepherd, "Statlog (Vehicle Silhouettes)," UCI Machine Learning Repository.
[42] A. Krizhevsky, "Learning multiple layers of features from tiny images," *Tech. Rep.*, 2009.
[43] N. Ilter and H. Guvenir, "Dermatology," *UCI Mach. Learn. Repository*, 1998.
[44] X. Huang, J. Li, and X. Hu, "Label informed attributed network embedding," in *Proc. 10th ACM Int. Conf. Web Search Data Mining*, 2017, pp. 731–739.
[45] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 22118–22133.
[46] H. Li, W. Xu, C. Qiu, and J. Pei, "Fast Markov clustering algorithm based on belief dynamics," *IEEE Trans. Cybern.*, vol. 53, no. 6, pp. 3716–3725, Jun. 2023.
[47] F. Lin and W. W. Cohen, "Power iteration clustering," in *Proc. 27th Int. Conf. Int. Conf. Mach. Learn.*, 2010, pp. 655–662.
[48] K. Jiang, Z. Liu, L. Zhu, and L. Cui, "Graph embedded subspace clustering with entropy-based feature weighting," *Int. J. Mach. Learn. Cybern.*, vol. 16, no. 9, pp. 5727–5745, 2025.
[49] F. Bi, T. He, Y.-S. Ong, and X. Luo, "Graph linear convolution pooling for learning in incomplete high-dimensional data," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 4, pp. 1838–1852, Apr. 2025.
[50] T. He, Y. S. Ong, and L. Bai, "Learning conjoint attentions for graph neural nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 2641–2653.
[51] T. He, Y. Liu, Y.-S. Ong, X. Wu, and X. Luo, "Polarized message-passing in graph neural networks," *Artif. Intell.*, vol. 331, 2024, Art. no. 104129.
[52] A. Ahmad and S. S. Khan, "initKmix-A novel initial partition generation algorithm for clustering mixed data using k-means-based clustering," *Expert Syst. Appl.*, vol. 167, 2021, Art. no. 114149.
[53] Z. Yang, F. Yu, W. Pedrycz, H. Yang, Y. Tang, and C. Ouyang, "A trend-granulation-based fuzzy C-means algorithm for clustering interval-valued time series," *IEEE Trans. Fuzzy Syst.*, vol. 32, no. 3, pp. 1263–1277, Mar. 2024.
[54] C. Liu, J. Guo, X. Zhang, D. Wu, and L. Yu, "Expert credibility prediction model based on fuzzy C-means clustering and similarity association," *IEEE Trans. Fuzzy Syst.*, vol. 33, no. 8, pp. 2719–2729, Aug. 2025.

Wei Chen received the bachelor's degree in civil engineering from the Harbin Institute of Technology, in 2009, and the master's degree and the PhD degree in electronic and communication engineering from Politecnico di Torino, Italy, in 2011 and 2016, respectively. He is currently a lecturer with the School of Software and Internet-of-Things Engineering, Jiangxi University of Finance and Economics, China. His research interests include signal processing and pattern analysis fields

Xiaohui Liu received the BSc degree in mathematics and applied mathematics from the Changsha University of Science & Technology, Changsha, China, in 2006, and the PhD degree in statistics from Central South University, Changsha, in 2012. His research interests include robust statistics and time series analysis.

Man Qiu (Graduate Student Member, IEEE) is currently working toward the MS degree in integrated circuit science and engineering with the School of Integrated Circuits, Southeast University, Nanjing, China. His research interests include data processing, neural networks, embedded systems, and integrated circuits.

Zhenning Xiong is currently working toward the master's degree with the School of Software and Internet of Things Engineering, Jiangxi University of Finance and Economics. His research interests include graph clustering and LLM.